Lecture 17: 4th March 2026

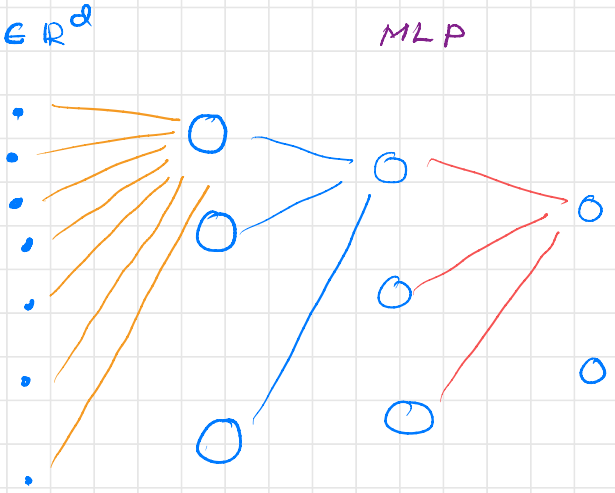
Convolutional Neural Networks:

→ Regularized MLP suited for "grid like" topological data

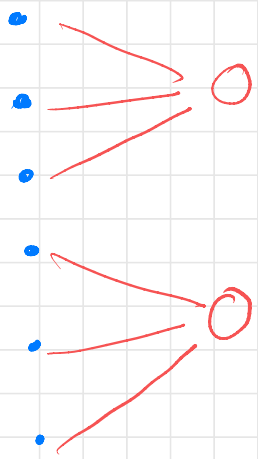
→ Eg: Image, 3D volumes

CNN is an MLP with a specific "hand" regularizer on the parameter space.

1) Local receptive field
2) Parameter sharing.

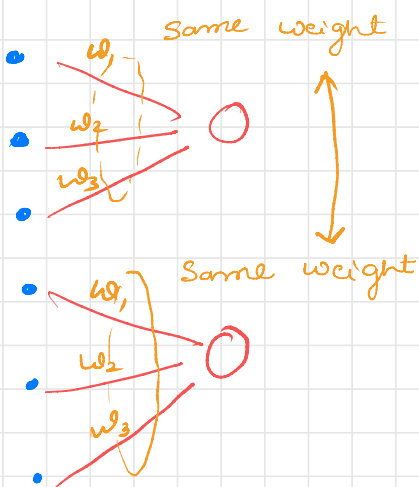


→ We will connect only subset of previous neuron to next layers



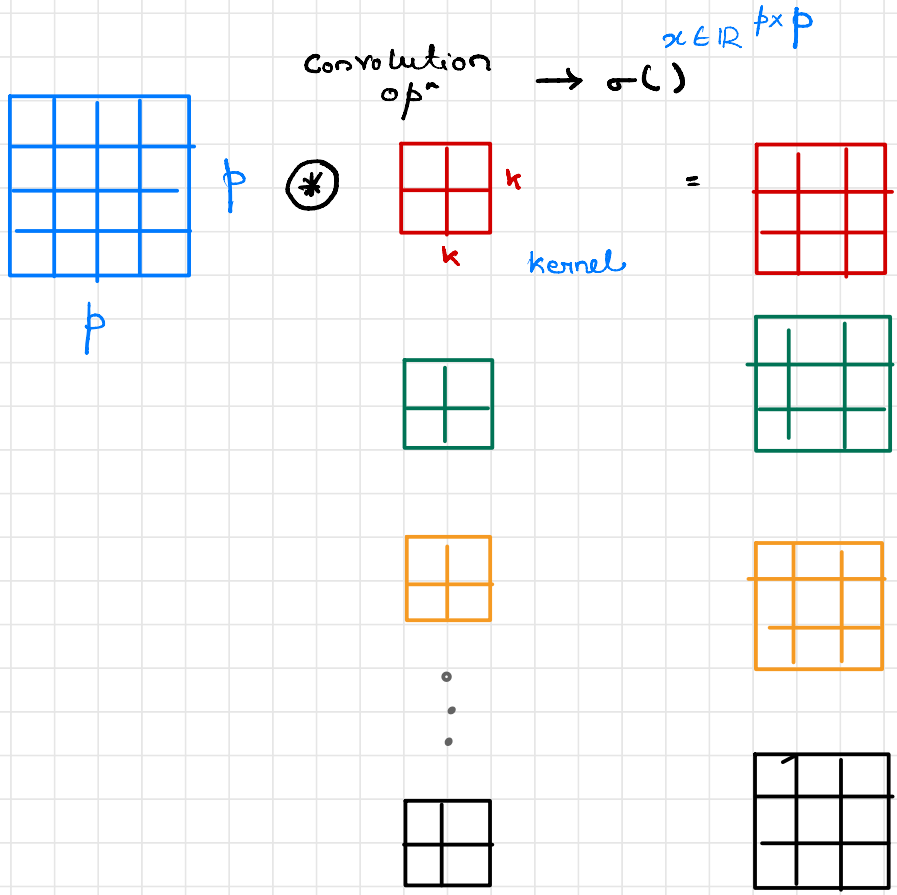
Local receptive field.
Connect only a subset of data / activations to every neuron in each layer.

Parameter sharing :



parameters of every receptive field is kept the same

Backprop in CNNs, do it in Tutorial.

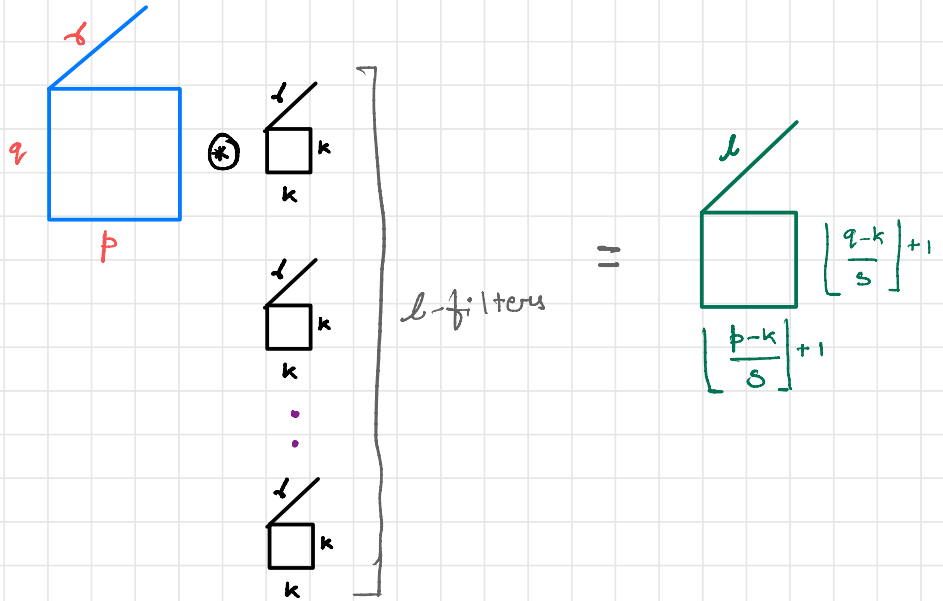


→ we hoover this over the whole image.

→ how much to move : stride

→ The operation is taking the inner product & add
This is the Convolution operation.

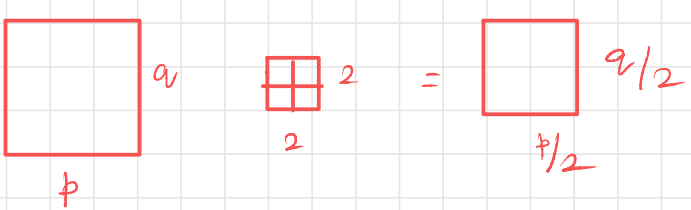
→ how many filters is again another parameters.



Now the hyper parameters

1) Num of filters in each layer
2) Filter Size
3) Stride & padding.

* Pooling :

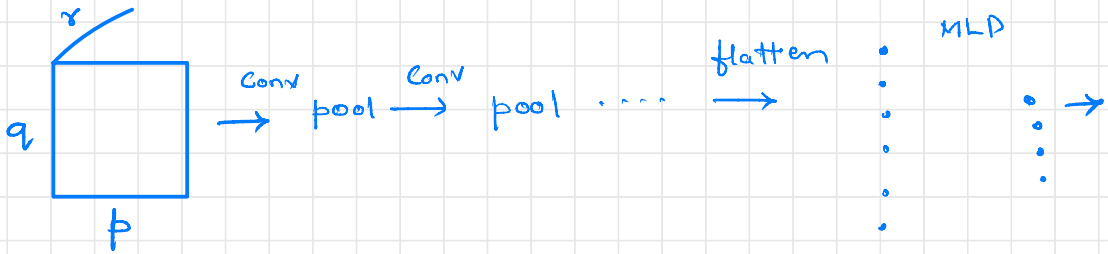


Avg (or) Max pooling.

Hard coding is done for Back prop

Examples of CNNs End to End:

1) Classification:



Then you do ERM

Le net

Alexnet

Res net

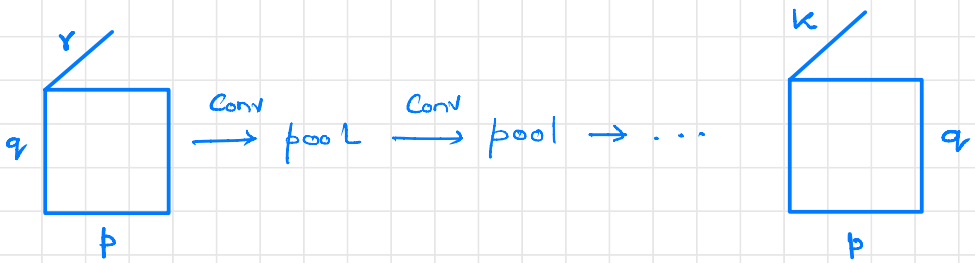
Inception net

ii) Fully Convolutional:

Semantic Segmentation.

input is image

output is image.



k: number of class.

Unet:

Encoder decoder Architecture.

Lecture 18: Recurrent Neural Networks :

Suited for Sequential Data

Consider a data point to be of following nature

$x = \{ x_1, x_2, \ldots, x_t \}$: Single Data point

where $x_t \in \mathbb{R}^d$

eg: natural language

clinical time series.

Video

Example task on such data:

i) Sequence classification.

Emotion extraction from text

ii) Sequence to Sequence Regression task

Machine translation.

* In Sequential data, every data point may be of diff length. and therefore MLP can't handle it

* In RNNs there is parameter sharing across "time"

* Consider one single data point

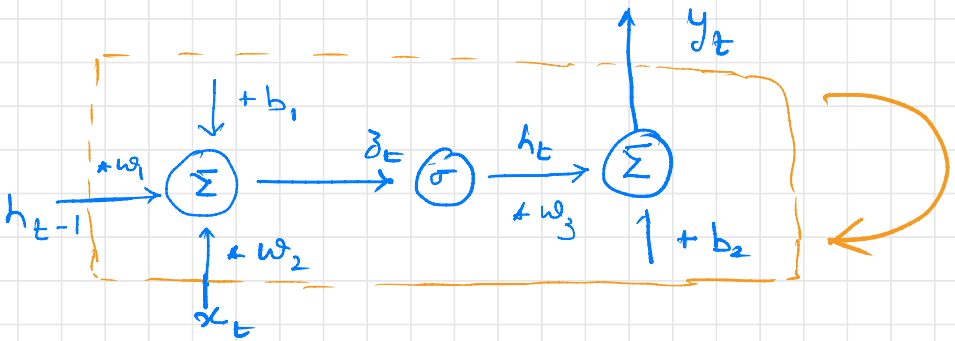
$$x = \{ x_1, x_2, \ldots, x_T \} \qquad x_t \in \mathbb{R}^d$$

$$z_t = w_1 h_{t-1} + w_2 x_t + b_1$$

$$h_t = \sigma(z_t)$$

$$\hat{y}_t = w_3 h_t + b_2$$

Single RNN Cell



→ Encoder Decoder model for Machine Translation.

→ we can stack them one top of another and give the idea of depth.

Back-propagation in RNNs.

→ Gradients will have 2 paths:

Right to left & top to bottom.

* when we do this its knowns as BPTT

Back propogation through Time.

Do it in tutorials.

* BPTT leads to vanishing gradients in long sequences.

Vanishing gradients in RNNs:

At each time step "t"

$$z_t = w_1 h_{t-1} + w_2 x_t$$

$$h_t = \sigma(z_t)$$

Suppose L_T be the loss at time T.

$$\frac{\partial}{\partial h_t} L_T = \frac{\partial}{\partial h_T} L_T * \frac{\partial}{\partial h_t} h_T$$

Consider

$$\frac{\partial}{\partial h_t} h_T = \prod_{k=t}^{T-1} \frac{\partial}{\partial h_k} h_{k+1}$$

Consider

$$\frac{\partial}{\partial h_k} h_{k+1}$$

we have

$$h_{k+1} = \sigma(w_1 h_k + w_2 x_{k+1})$$

$$\frac{\partial}{\partial h_k} h_{k+1} = \text{diag}\left(\sigma'(z_{k+1})\right) w_1$$ Show in tutorial.

Consider the norm of the derivative. $\frac{\partial}{\partial h_t} h_T$

By Cauchy - inequality (CS inequality).

$$\left\| \frac{\partial}{\partial h_t} h_T \right\| \leq \prod_{k=t}^{T-1} \left\{ \underbrace{\left\| \text{diag}\left(\sigma'(z_{k+1})\right) \right\|}_{\leq 1} \|w_1\| \right\}$$

Let λ_{max} be the largest singular value of w_1

$$\|w_1\| = \lambda_{max}.$$

$$\left\| \frac{\partial}{\partial h_t} h_T \right\| \leq (\lambda_{max})^{T-t}$$

as the gap $T-t \uparrow$ $(\lambda_{max})^{T-t}$ decays exponentially. ⓞ explodes depending on the value of λ_{max}.

The General Mathematical objective.

In a recurrent system:

$$h_t = F(h_{t-1}, x_t)$$ Composite of linear / non linear.

$$\frac{\partial}{\partial h_t} h_T = \prod_{k=t}^{T-1} \frac{\partial}{\partial h_k} h_{k+1}$$

In a Vanilla RNN, vanishing grad happens since.

$$\frac{\partial}{\partial h_k} h_{k+1} = \text{diag}(\sigma'(z_{k+1})) \cdot W$$

Solu^n is to make

$$\frac{\partial}{\partial h_k} h_{k+1} \simeq I$$

To achieve this, the state update should be additive

$$h_t = \alpha_t \odot h_{t-1} + \beta_t \odot \tilde{h}_t$$

where h_{t-1} is the previous state &

$$\tilde{h}_t = \sigma(w_2 x_t + w_1 h_{t-1})$$

α_t & β_t are "modulators"

$\beta_t = 1 \Rightarrow$ highway network

$\beta_t = 1 - \alpha_t \Rightarrow$ GRU

α_t & β_t are learnt $\Rightarrow$ LSTM

The gradients for the above networks

Consider

$$\frac{\partial}{\partial h_{t-1}} h_t = \text{diag}(\alpha_t) + \frac{\partial}{\partial h_{t-1}} \alpha_t \, \text{diag}(h_{t-1}) + \frac{\partial}{\partial h_{t-1}} (\beta_t \odot \tilde{h}_t)$$

$$= \text{diag}(\alpha_t) + \epsilon_t(w_1, w_2)$$

now

$$\frac{\partial}{\partial h_t} h_T = \prod_{k=t}^{T-1} \left(\text{diag}(\alpha_{k+1}) + \epsilon_{k+1} \right)$$

now CS inequality

$$\left\| \frac{\partial}{\partial h_t} h_T \right\| \leq \prod_{k=t}^{T-1} (\bar{1}) = 1$$